SSD project Overview

William Lecky-Thompson (000613648), Taha Ahmed (000613520)

January 18, 2026

Contents

1 Framework Selection: Django **3**
1.1 Built-in Security Features . . . 3
1.1.1 Cross-Site Scripting (XSS) Protection . . . 3
1.1.2 Cross-Site Request Forgery (CSRF) Protection . . . 3
1.1.3 SQL Injection Prevention . . . 3
1.1.4 Clickjacking Protection . . . 3
1.1.5 SSL/HTTPS Support . . . 3
1.1.6 Session Security . . . 3
1.1.7 Security Middleware . . . 4
1.2 Additional Security Integrations . . . 4
1.3 Implementation . . . 4

2 WebAuthn Authentication **4**
2.1 WebAuthn Data Model . . . 4
2.2 Registration Flow . . . 4
2.3 Login Flow . . . 5
2.4 Doctor Certificate-Based Authentication . . . 5
2.4.1 Certificate Issuance . . . 5
2.4.2 Authentication Enforcement . . . 5
2.4.3 Certificate Renewal and Expiration Handling . . . 6
2.4.4 Security Rationale . . . 6
2.5 Device Management . . . 6
2.6 Security Considerations . . . 6

3 End-To-End Encryption **6**
3.1 End-To-End Setup . . . 6
3.1.1 Device Authentication . . . 6
3.1.2 Authentication Scenarios: . . . 7
3.1.3 Device managment . . . 7
3.2 End-To-End Implementation . . . 7
3.2.1 File Upload: . . . 8
3.2.2 File Edit: . . . 8
3.2.3 File Delete: . . . 8
3.2.4 Patient-Doctor Relationship: . . . 8

4 Audit Logging Architecture **8**
4.1 Two-Server Design . . . 8
4.2 MetaData . . . 9
4.3 Log Types . . . 9
4.4 Cryptographic Flow . . . 9
4.4.1 Backend Process . . . 9
4.4.2 Security Server Process . . . 9
4.5 Data Storage Format . . . 10
4.6 Hash Chain Construction . . . 10
4.6.1 Hash Calculation . . . 10
4.6.2 Chain Structure . . . 10
4.6.3 HSM Signature Binding . . . 10
4.7 Chain Integrity Verification . . . 11

4.7.1 Verification Process . . . 11
4.7.2 Tamper Detection . . . 11
4.8 Hardware Security Module (HSM) . . . 11
4.8.1 SoftHSM Implementation . . . 11
4.8.2 Key Operations . . . 11
4.9 Security Properties . . . 12
4.9.1 Separation of Duties . . . 12
4.9.2 Confidentiality . . . 12
4.9.3 Integrity . . . 12
4.9.4 Non-Repudiation . . . 12
4.9.5 Tamper Evidence . . . 12

1 Framework Selection: Django

For this project, we chose Django, a high-level Python web framework, primarily due to its comprehensive built-in security features and strong security-by-default philosophy. Django follows a "secure by default" design philosophy, where security features are enabled and properly configured out of the box, rather than requiring developers to manually implement them. This approach significantly reduces the risk of security vulnerabilities resulting from misconfiguration or oversight, which are common causes of web application breaches.

1.1 Built-in Security Features

Django provides a comprehensive suite of security features that protect against the most common web application vulnerabilities as identified by the OWASP Top 10. The following subsections detail these security measures and their implementation in our application.

1.1.1 Cross-Site Scripting (XSS) Protection

Django's template system automatically escapes all variables by default, preventing malicious scripts from being injected into web pages. This automatic escaping applies to special characters such as `<`, `>`, `'`, `"`, and `&`, converting them to their HTML entity equivalents. The framework provides the `mark_safe()` function and `|safe` filter for cases where unescaped content is intentionally required.

1.1.2 Cross-Site Request Forgery (CSRF) Protection

Django includes middleware that protects against CSRF attacks by requiring a secret token for all POST, PUT, PATCH, and DELETE requests. The `CsrfViewMiddleware` generates unique tokens for each user session and validates them on form submissions, ensuring that requests originate from the legitimate application rather than malicious third-party sites. Our implementation includes CSRF trusted origins configuration to support HTTPS connections and proper cross-origin request handling.

1.1.3 SQL Injection Prevention

The Django Object-Relational Mapper (ORM) automatically parameterizes all database queries, effectively preventing SQL injection attacks. By using prepared statements and query parameterization, Django ensures that user input is never directly concatenated into SQL queries. The ORM's query construction methods (`filter()`, `exclude()`, `get()`, etc.) handle escaping and quoting automatically, providing a safe abstraction layer over raw SQL.

1.1.4 Clickjacking Protection

Django's `XFrameOptionsMiddleware` protects against clickjacking attacks by setting the `X-Frame-Options` HTTP header. This prevents malicious sites from embedding our application in iframes and tricking users into performing unintended actions. The framework supports configuring this header to `DENY` (preventing all framing) or `SAMEORIGIN` (allowing framing only from the same origin).

1.1.5 SSL/HTTPS Support

Django provides comprehensive support for HTTPS deployments through multiple security settings. The `SECURE_SSL_REDIRECT` setting automatically redirects all HTTP requests to HTTPS, ensuring encrypted communication. The framework supports HTTP Strict Transport Security (HSTS) through the `SECURE_HSTS_SECONDS` setting, which instructs browsers to only communicate with the server over HTTPS for a specified duration. Additional settings like `SECURE_HSTS_INCLUDE_SUBDOMAINS` and `SECURE_HSTS_PRELOAD` provide enhanced protection across the entire domain hierarchy.

1.1.6 Session Security

Django's session framework includes multiple security features to protect user sessions from hijacking and fixation attacks. Session cookies can be configured with the `HttpOnly` flag to prevent JavaScript access, reducing the risk of session token theft through XSS attacks. The `Secure` flag ensures cookies are only transmitted over HTTPS connections. The framework supports configurable session expiration through `SESSION_COOKIE_AGE` and can be configured to expire sessions when the browser closes using `SESSION_EXPIRE_AT_BROWSER_CLOSE`.

1.1.7 Security Middleware

Django's `SecurityMiddleware` provides several HTTP header-based protections. It sets the `X-Content-Type-Options` header to prevent MIME type sniffing attacks, adds the `X-XSS-Protection` header for additional XSS protection in older browsers, and manages SSL-related headers. The middleware also handles secure proxy header processing for applications deployed behind reverse proxies.

1.2 Additional Security Integrations

Beyond Django's built-in features, our implementation incorporates additional security measures including:

- **PKI Infrastructure**: Integration of Public Key Infrastructure with root, intermediate, and server certificates for mutual TLS authentication
- **Certificate Verification Middleware**: Custom middleware for validating client certificates and enforcing certificate-based authentication
- **WebAuthn Integration**: Support for FIDO2-compliant passwordless authentication using biometric or hardware security keys
- **WhiteNoise**: Secure static file serving with compression and caching headers
- **Database Connection Security**: Encrypted MySQL connections with SSL/TLS support
- **Security Logging**: Comprehensive logging of security-relevant events for audit and intrusion detection

1.3 Implementation

To properly develop this project, we decided to dockerize the project. We have four different services for this application:

- Web Service: The Django application
- Database Service: A container running mysql, our choice for the database.
- SecurityLogger Service: Another Django application acting as another server for storing a copy of the security logs
- Nginx Service: Handles serving the project with SSL implementation and certificate verification

2 WebAuthn Authentication

The application implements a passwordless authentication mechanism based on the WebAuthn standard. Users authenticate by proving possession of a cryptographic authenticator (e.g., hardware security key, mobile device, or platform authenticator), eliminating the need for traditional passwords.

Two user groups are supported: **Patients** and **Doctors**. While both groups rely on WebAuthn for authentication, Doctors are subject to additional certificate-based verification.

2.1 WebAuthn Data Model

The system defines a single persistence model, `WebAuthDevice`, which represents a registered WebAuthn credential. This model stores only public information, including the credential identifier, public key, signature counter, and associated metadata. No private keys or sensitive secrets are stored server-side; therefore, encryption at rest is not required.

A user may register multiple `WebAuthDevice` entries, enabling authentication from multiple devices.

2.2 Registration Flow

During registration, a user provides a username, selects a role (Patient or Doctor), and assigns a name to the authentication key. No password is requested, as authentication relies exclusively on possession of the WebAuthn credential.

The registration ceremony follows the standard WebAuthn flow:

1. The frontend submits the username, role, and key name to the backend.

2. The backend generates a WebAuthn challenge and returns the public key creation parameters.
3. The frontend invokes `navigator.credentials.create()` to generate a credential.
4. The generated public credential is sent back to the backend.
5. The backend verifies the challenge and attestation data.
6. A new user is created, the appropriate role is assigned, and a `WebAuthDevice` record is stored.

If the user is a Doctor, the backend additionally issues a client certificate. The certificate is delivered as a password-protected PKCS#12 bundle, and the password is displayed only once. The certificate must be installed in the browser; otherwise, registration and subsequent login attempts will fail.

2.3 Login Flow

To authenticate, a user submits only a username. The backend generates a short-lived authentication token and, upon validation, issues a WebAuthn challenge along with the user's registered credential identifiers.

The frontend calls `navigator.credentials.get()`, prompting the authenticator to sign the challenge. The backend verifies the resulting assertion by validating the embedded challenge, the cryptographic signature against the stored public key, and the signature counter to prevent replay attacks.

For Doctors, the backend additionally validates the client certificate and its trust chain. If the certificate is missing, invalid, or expired, authentication is denied.

2.4 Doctor Certificate-Based Authentication

In addition to WebAuthn authentication, Doctors are required to authenticate using a client-side X.509 certificate. This mechanism provides strong, cryptographically verifiable identity assurance and enforces a form of mutual TLS at the application layer.

2.4.1 Certificate Issuance

Upon successful registration of a Doctor account, the backend issues a personal client authentication certificate. The certificate is generated using an internal Public Key Infrastructure (PKI) and is signed by an intermediate Certificate Authority (CA), which in turn is anchored to a trusted root CA.

The issued certificate has the following properties:

- An RSA key pair generated server-side with a 2048-bit modulus.
- A subject containing the Doctor's unique username and role metadata.
- An `ExtendedKeyUsage` extension restricted to `ClientAuth`.
- A bounded validity period (default: 365 days).

The private key and certificate are delivered to the Doctor as a password-protected PKCS#12 bundle. The password is generated randomly and displayed only once at issuance time. The Doctor must install the certificate in their browser or operating system trust store. The server does not retain any copy of the private key.

Certificate metadata (serial number, issuance time, expiration time, and revocation status) is stored in the database for lifecycle management and auditing purposes.

2.4.2 Authentication Enforcement

During login, after successful WebAuthn assertion verification, the backend enforces certificate-based authentication for Doctor accounts. The client certificate is extracted from the incoming HTTPS request and validated before a session is established.

The validation process includes:

1. Verification that a client certificate is present.
2. Cryptographic verification of the certificate signature against the intermediate CA.
3. Verification of the intermediate CA signature against the root CA.
4. Validation of certificate validity periods (client, intermediate, and root).
5. Enforcement of the `ClientAuth` extended key usage.

If any of these checks fail, authentication is denied and no session is created, even if the WebAuthn verification succeeded.

2.4.3 Certificate Renewal and Expiration Handling

Doctor certificates are subject to expiration. During authentication, the system checks whether the most recent non-revoked certificate associated with the Doctor account is expired or nearing expiration.

If a certificate is expired or within a configurable renewal window, authentication is interrupted and a short-lived renewal token is issued. This token allows the Doctor to securely request a new certificate without re-registering their account. Previously valid certificates are marked as revoked once a new certificate is issued, ensuring that only a single active certificate is accepted at any given time.

2.4.4 Security Rationale

By combining WebAuthn with certificate-based authentication, the system enforces a dual-factor model based entirely on cryptographic possession:

- WebAuthn proves possession of a registered authenticator bound to the user.
- The client certificate proves possession of a role-specific, CA-issued credential.

This layered approach significantly raises the security bar for Doctor accounts, mitigates phishing and credential replay attacks, and provides strong guarantees of professional identity without relying on passwords or shared secrets.

2.5 Device Management

Authenticated users may register additional WebAuthn devices. This process mirrors the registration ceremony but does not involve user creation or role assignment. It assumes the user already possesses at least one trusted device (typically a mobile phone) capable of securely storing authentication keys.

2.6 Security Considerations

The WebAuthn challenge is embedded within `clientDataJSON` and is not signed directly. Challenge validation, signature verification, and authenticator metadata checks are performed together during assertion verification. Temporary challenges and tokens are invalidated immediately after successful authentication to mitigate replay attacks.

3 End-To-End Encryption

This section establishes how we implemented End-To-End Encryption to handle everything from file upload to file edit and review.

3.1 End-To-End Setup

For this implementation, two things need to be setup to enable End-To-End Encryption: Master keys and device. Every user has a set of two asymmetric key pairs, one for encryption and one for signing. These keys called master keys, and are the essential component of the whole project. They enable everything from file upload, to file modification and signature. They are generated after user registration. If these keys are lost, we lose access to everything. These keys are then encrypted using a symmetric key generated from a random string, called recovery string. This random sequence of characters is shown once to the patient, who must save it safely in order to be able to recover his master keys in case of new device login.

This recovery string is generated using the *crypto* library's *getRandomValues* function. This will generate a 32-character random string to ensure there is no easy way to guess it.

Once the recovery string is generated and saved, and the master keys encrypted, the fundamentals are set. If the user wants to connect to the platform using a new device, he or she must provide the recovery string, from which a symmetric key capable of decrypting the master keys is derived, and the user gains access once again to his files.

3.1.1 Device Authentication

We do not need the user to enter his recovery string each time he logs into his account from the same device, as it might introduce vulnerabilities where the user simply writes the recovery string next to his device for everyone to see, and it would me very annoying in term of user experience. To solve this problem, we have decided to use device authentication. The way it works is very simple:

Once the master keys are setup, the user is redirected to a device setup page, where a symmetric key is generated for the current device. This symmetric key also encrypts the master keys, meaning the master keys are saved encrypted twice: once with the recovery string, and another time with the device key and identified by the device.

The device key itself is not saved in the server however, it is only saved in the device in a secure manner (client-side only). We say device, but in reality it is saved in the browser, more specifically in the browser's indexedDB. The device key is generated with the extractable option set to false, meaning it cannot be exported, which adds a security layer to the key.

When the user logs into his account from a known device that was set up, we get the user's master keys from the database, more specifically the ones that are encyrpted with the device key, and decrypt them on the client using the device key. Then, the user can use the application.

3.1.2 Authentication Scenarios:

In summary, here are the different scenarios that can happen once the user successfully logged in to his account using webauthn:

- **First time login (no keys and no device)**: The user is redirected to the keys setup page where a 32-character recovery string is generated, a derived symmetric key from said string is also generated, along with the master keys. The master keys are then encrypted using this derived key and stored on the server. The derived key is never stored. The user now has a set of master keys for encryption and decryption.
- **Login from a new device (master keys setup)**: The user has his master keys setup, but the device the user used to log in is not recognized (no device key or no device-encrypted master keys). In this case, the user is redirected to a device setup page, where the user gives his device a friendly name to recognise it and manage it later on, then, if the master keys were setup in the same session, then the user does not have to provide teh recovery string, otherwise it must be provided to recover the master keys. Then a device key is generated, the master keys are encrypted with it and stored in the server, and the device key is stored on the client.
- **Login from a known device**: The user has setup his master keys and the device. In this case, the user's device-encrypted master keys are retrieved, and the client decrypts them using the device key, and proceed to the app.

3.1.3 Device managment

The user can see the list of devices he used to log in to his account, and can learn information about them such as their IP address and last logged in on. The user can also revoke a device, in which case the master keys that were encrypted using the device's key are deleted, making the user unable to access his keys from that device without setting it up again (and provide the recover string).

The use of device-key for master key encryption makes the login process easy for the user in term of user experience, without compromising security. Device managment and revokation adds more security control to the process by enabling the user to revoke access to compromised devices for example. The keys are stored in the browser, meaning they are OS-independant, enabling the user to login from different kind of devices and operating systems (Windows, Android, Linux) with no problem (provided they have the recovery string of course).

3.2 End-To-End Implementation

Now that the user has his set of master keys, he or she can use them to work with the medical files.

The application recognizes two kinds of users: Patients and Doctors. Each patient has a repository for his medical files. The patient can freely upload new files to his repository, edit them and delete them.

The patient also has a list of appointed doctors, users recognized as doctors that are granted access to the patient's files. These doctors can see the list of medical files of their patients, but in order to do any action on them, such as making modifications to their content, deleting them, or even uploading new files, they must get explicit permission from the patient.

The following sections describe how these file actions are handled in both sides in a secure manner to ensure the server cannot see or act on these files, only the authorized users can.

3.2.1 File Upload:

When a patient decides to upload a new file to his repository, he or she selects the file from their device. A symmetric key is then generated for this file, and used to encrypt the file content and file name. These encrypted data are then signed using the patient's signing key (private key). A grant is then created for the patient himself and all of his appointed doctors. A user's grant includes the the file key (symmetric) encrypted with the user's public key, and the encrypted filename for quick access. This grant is what allows the patient and his appointed doctors to access the file and act on it. The encrypted file content, name and metadata, such as mime-type, size etc. are sent to the server along with the access grant for each authorized user. The server stores these data in a specific manner: The basic file information, such as the file's metadata, name, and timestamps are saved in a MedicalFile table. The file content and signature however, are stored in another table called MedicalFileVersion. Indeed, we have file versioning where each edit makes a new version instead of simply overwriting the file itself. The first version of a file is the version created when the file is first uploaded, and each subsequent edit creates a new version from the edited version. For each version we store the file it references, the previous version to keep track of edit timeline, the encrypted content, the content size, the signature of the editor and the status of the version (approved, pending, declined).

When a doctor wants to upload a new file for a patient, the process is the same, except that the file version status is set to pending, and only the patient and the uploader doctor has access to the file, not the other doctors appointed to the patient. Only when the patient has approved the upload would the access be shared to the other patients. A pending upload file cannot be edited until it is approved. If the patient decides to decline the upload, the file and all of its versions and grants get deleted along with it.

3.2.2 File Edit:

When a patient decides to edit the content of a file, he can only edit the content of the latest approved version. Once the edits are made and ready for submission, a new symmetric key is generated for the new version, and this key encrypts the new content. This means we have an encryption key per version, which allows us to grant access to specific versions if we need to, but also enables **forward-secrecy** because a compromised version key will not compromise the safety of other versions. The new content is then signed along with the previous version's signature and these data are sent to the server for encryption. Each authorized user is then granted access to this new version the same way we did during file upload. This way of signing a new version enables signature chaining, where the authenticity and integrity of a version relies on the authenticity and integrity of all previous versions. When a doctor wants to edit the content of a file, he makes the edit and requests patient approval. Once the patient approves the edit, a new version is created from this edit and all other pending edits are declined. This prevents the versions from branching out and keeps it linear.

3.2.3 File Delete:

File versions cannot be deleted once approved, only the entire file can be deleted. A patient can delete his files whenever he wants, and all of its related data are deleted from the server, including the version keys and therefore the grants. A doctor can request a file deletion, in which case it is set to pending deletion, and the patient needs to approve its deletion, until then, the file is accessible to all authorized users.

3.2.4 Patient-Doctor Relationship:

Each patient has access to the list of all patients registered in the platform, can can appoint whoever he wants to his medical record without their explicit permission. Once appointed, the doctor receives access to all existing patient files and their versions. This is done by the patient on the client-side, who will decrypt all file version keys and encrypt them with the new doctor's public key. Each patient can also remove a doctor from his list of appointed doctor, in which case that doctor's grants get deleted from the server, revoking his access to all patient files.

Each doctor also sees a list of all registered patients, and can make a request to a patient to be appointed or un-appointed. The patient then decides to accept or decline the request.

4 Audit Logging Architecture

4.1 Two-Server Design

The system uses separate servers with distinct administrators:

- **Hospital Backend (Admin A)**: Generates logs, encrypts security logs, cannot read encrypted logs
- **SecurityLogger (Admin B)**: Stores encrypted logs with HSM signatures, cannot forge logs

This separation ensures no single administrator can tamper with security logs undetected.

4.2 MetaData

Each log includes:

- log action
- timestamp
- response status
- response state (ok, error)
- http method
- http route
- username/user id (if applicable)
- user agent
- duration of the request/response
- additional data relevant to the request such as file size, reciepientCount, mimetype, etc.

4.3 Log Types

Standard Logs: Stored locally on the backend for debugging and system monitoring.

Security Logs: Encrypted and stored on both servers. Track authentication, authorization, data access, and admin actions.

4.4 Cryptographic Flow

4.4.1 Backend Process

The logs are encrypted client-side. Each log is encrypted with a newly generated symmetric key to handle variable length logs. What is sent to the servers is then the encrypted logs, the symmetric key encrypted with the proper server's public key, and additional cryptographic data such as the IV, and tag for validation. When a security event occurs:

1. Assign sequential sequence number
2. Create structured payload: {`sequence_number, payload_data`}
3. Compute HMAC-SHA256 on the encrypted bytes using shared secret
4. Base64-encode encrypted payload and HMAC
5. Send to Security Server: {`payload, backend_signature, sequence_number`}
6. Store encrypted payload and server response locally

Note: The sequence number appears both inside the encrypted payload and as a separate field. The external sequence number is used for validation and chain construction, while the encrypted copy provides tamper evidence for the payload itself.

4.4.2 Security Server Process

Upon receiving an encrypted log:

1. Verify HMAC-SHA256 signature on encrypted payload
2. Verify sequence number is sequential (no gaps, no duplicates)
3. **Verify hash chain integrity of existing entries**
4. **Halt processing if chain is broken (critical security check)**
5. Calculate previous_hash: SHA256 of complete JSON-serialized previous entry

6. Create signing data structure: {`sequence_number, payload, previous_hash`}
7. Compute SHA256 digest of JSON-serialized structure
8. Sign digest using HSM private key (RSA-PKCS#1 v1.5 with SHA256)
9. Store: encrypted payload (base64 string), signatures (binary), previous_hash (hex string)
10. Return HSM signature and current hash to backend

Important: The Security Server stores encrypted payloads without decrypting them, maintaining end-to-end confidentiality.

4.5 Data Storage Format

Each log entry is stored with specific encoding for each field:

- **sequence_number**: Integer
- **payload**: Base64-encoded string of encrypted data
- **backend_signature**: Binary blob (HMAC bytes)
- **server_signature**: Binary blob (RSA signature bytes)
- **previous_hash**: Hexadecimal string (64 characters)

4.6 Hash Chain Construction

4.6.1 Hash Calculation

For each entry, the hash is computed over a JSON-serialized structure with sorted keys:

```
entry_data = json.dumps({
    "sequence_number": <int>,
    "payload": <base64_string>,
    "backend_signature": <base64_string>,
    "server_signature": <base64_string>,
    "previous_hash": <hex_string>
}, sort_keys=True)

current_hash = SHA256(entry_data.encode('utf-8'))
```

This produces a 256-bit hash represented as a 64-character hexadecimal string.

4.6.2 Chain Structure

Each entry links to its predecessor:

- Entry 1 (Genesis): previous_hash = `"0000...0000"` (64 zeros)
- Entry N: previous_hash = SHA256(complete Entry N-1 data)

4.6.3 HSM Signature Binding

The HSM signature cryptographically binds each entry to the chain:

1. Create signing structure: {`sequence_number, payload, previous_hash`}
2. Serialize to JSON with sorted keys
3. Compute SHA256 digest of serialized data
4. HSM signs the digest using RSA private key with SHA256-RSA-PKCS mechanism

The signature covers the encrypted payload, sequence number, and previous_hash. This ensures any modification to the current entry or any previous entry invalidates all subsequent signatures.

4.7 Chain Integrity Verification

4.7.1 Verification Process

Before accepting any new log entry, the Security Server:

1. Retrieves the last stored entry
2. Computes the hash of the last entry
3. Verifies it matches the stored previous_hash
4. **REJECTS the new entry if verification fails**

Optionally, full chain verification can be performed:

1. Iterate through all entries in sequence order
2. For each entry, verify its previous_hash matches the computed hash of the prior entry
3. Report any breaks in the chain

4.7.2 Tamper Detection

If any historical entry is modified:

- The next entry's previous_hash won't match the modified entry's hash
- Chain verification detects the break immediately
- The HSM signature becomes invalid (it signed the original previous_hash)
- **All new entries are rejected until integrity is restored**
- The exact location of tampering is identified

4.8 Hardware Security Module (HSM)

4.8.1 SoftHSM Implementation

SoftHSM provides PKCS#11-compliant HSM functionality:

- Private keys stored in encrypted token files
- PIN protection (5678 in development) required for operations
- Private keys marked as non-extractable
- All cryptographic operations performed inside HSM library
- Administrators cannot access raw key material

4.8.2 Key Operations

Signing: The application provides data and PIN to the HSM. The HSM:

1. Computes SHA256 digest of the data (if not pre-hashed)
2. Signs the digest using the private key
3. Returns only the signature bytes

The private key never leaves the HSM token.

Key Generation: Keys are generated inside the HSM and never exist in extractable form. Only the public key is exported.

4.9 Security Properties

4.9.1 Separation of Duties

Hospital Admin A:

- Can generate security logs
- Cannot read security logs after encryption (no private key)
- Cannot access Security Server database
- Cannot modify sequence numbers without detection

Security Admin B:

- Cannot forge signatures (HSM-protected private key)
- Cannot modify history (chain breaks, processing halts)

4.9.2 Confidentiality

- Logs encrypted with RSA-2048 public key
- Security Server stores encrypted payloads without decryption
- Only HSM private key can decrypt (access requires PIN)
- Even Security Admin B cannot extract the private key

4.9.3 Integrity

- HMAC-SHA256 prevents tampering during transmission
- HSM RSA signature ensures authenticity
- Sequence numbers prevent gaps or reordering
- SHA256 chain prevents modification of history
- Chain verification before accepting new entries

4.9.4 Non-Repudiation

- Backend's HMAC proves log origin (shared secret)
- HSM signature proves server acceptance (private key)
- Both signatures are cryptographically verifiable
- Complete audit trail on both servers

4.9.5 Tamper Evidence

- Deleted entries create sequence gaps (rejected)
- Modified entries break the chain (processing halts)